Performance Testing with LoadRunner

1. What is Performance Testing?

- Performance Testing is a type of non-functional testing that checks how a system behaves under load, stress and real-world usage.

2. Why do we need it?

- To ensure system stability under load.
- To find performance bottlenecks before production.
- To ensure better user experience.
- To plan infrastructure capacity.

Real-world Example

An e-commerce website works fine for 10 users, but slows down or crashes when 1000 users try to buy during a sale. Performance Testing helps prevent this.

3. Functional Testing vs Performance Testing

Functional Testing	Performance Testing
• Checks WHAT the system does. • Validates features and functions. • Example: Login, Add to cart, Payment.	• Checks HOW the system performs. • Validates speed, stability, scalability. • Example: Response time, Load handling.

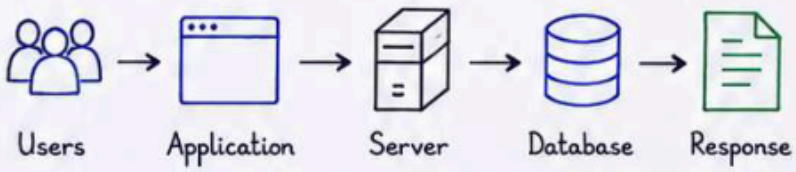
4. Performance Testing Objectives

- Validate system performance under expected load.
- Identify the breaking point.
- Ensure system meets SLA targets.
- Provide performance baseline for future comparison.

5. When should it be performed?

- After system is functionally stable.
- Before production release.
- After major code/infrastructure changes.
- Periodically to ensure performance standards.

6. Basic Performance Flow



2. Types of Performance Testing

Type	Purpose	What it Checks	Example
Load Testing	To check system behavior under expected load.	Response time, throughput, resource utilization.	1000 users shopping on a website.
Stress Testing	To check system behavior beyond normal capacity.	System stability at high load and breaking point.	Increasing users until system starts failing.
Spike Testing	To check system behavior for sudden load increase.	How system handles sudden spikes in load.	Traffic jumps from 100 to 1000 users suddenly.
Endurance / Soak Testing	To check system stability for long duration.	Memory leaks, performance degradation over time.	Running 500 users for 24 hours continuously.
Volume Testing	To check system behavior with large data volumes.	Performance with huge database or data load.	Testing with 10 million records in DB.
Scalability Testing	To check system scalability with increased resources.	System ability to scale with more resources.	Adding servers and checking improvement.
Capacity Testing	To determine system capacity limits.	Maximum users/transactions system can handle.	Finding max users before response time fails.
Baseline Testing	To establish baseline performance metrics.	Baseline for future comparisons.	Recording performance with 500 users.
Breakpoint Testing	To find the load where system breaks.	Exact point where performance collapses.	Identifying failure point under high load.

Key Takeaway

- ★ Different types of performance tests help us understand different aspects of system performance.
- ★ LoadRunner helps us simulate real-world scenarios and find performance issues before users do.

Quick Note

- ★ Load Testing → Expected Load
- ★ Stress Testing → Beyond Capacity
- ★ Spike Testing → Sudden Increase
- ★ Soak Testing → Long Duration
- ★ Capacity Testing → Find Max Limit

3. Performance Testing Terminology

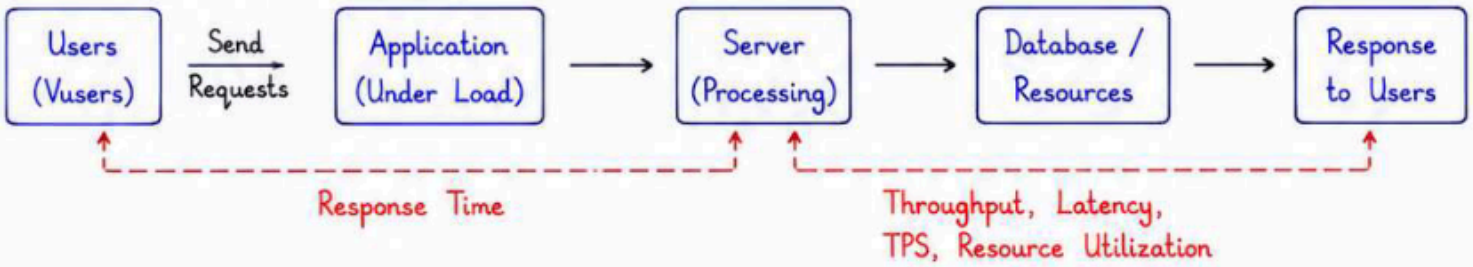
Term		Meaning
• Response Time	:	Total time taken from sending request to receiving complete response.
• Latency	:	Time taken for a single packet to travel from source to destination.
• Throughput	:	Amount of work done by the system in given time (e.g., requests/sec).
• TPS (Transactions Per Second)	:	Number of transactions processed per second.
• RPS (Requests Per Second)	:	Number of requests sent to the system per second.
• Concurrent Users (Vusers)	:	Number of virtual users executing at the same time.
• Active Users	:	Users who are performing transactions at any given time.
• Virtual Users (Vusers)	:	Simulated users generated by LoadRunner.
• Load	:	Number of users or transactions on the system.
• Peak Load	:	Maximum expected load during a specific time.
• Bandwidth	:	Amount of data transferred over a network in given time.
• Connection Time	:	Time taken to establish a connection.
• Processing Time	:	Time taken by the server to process the request.
• Queue Time	:	Time a request waits in queue before being processed.

Example

If 100 users send requests to a system and system gives 200 responses in 10 seconds.

- Throughput (RPS) = 200 / 10 = 20 requests/sec
- TPS depends on business transaction (e.g., login + search + add to cart = 1 transaction)
- Average Response Time = Total Response Time / Total Requests
- If Response Time increases with same load → Performance issue

How these terms relate ?



4. Performance Metrics & SLA

Important Performance Metrics

- Average Response Time : Average time taken to process a request.
- Min / Max Response Time : Minimum and maximum time for a request.
- Percentiles (P90, P95, P99) : 90%, 95%, 99% of requests completed within this time.
- Throughput : Number of transactions completed per second.
- Error Rate : Percentage of failed requests.
- CPU Utilization : CPU usage on the server.
- Memory Utilization : Memory used by the application / server.
- Disk I/O : Disk read/write usage.
- Network Utilization : Network bandwidth usage.
- Apdex (Application Performance Index) : User satisfaction metric.

Why Average is Misleading ?

Average response time can look OK even if some users are facing very slow response.

Example:

- Avg = 2 sec (looks good)
- But P95 = 8 sec
- Some users are experiencing poor performance.

Always check Percentiles (P95, P99) along with Average.

SLA, SLO, SLI

- SLA (Service Level Agreement) : Agreement between provider and customer.
- SLO (Service Level Objective) : Target set by provider to meet SLA.
- SLI (Service Level Indicator) : Measurable metric used to track SLO.

Example

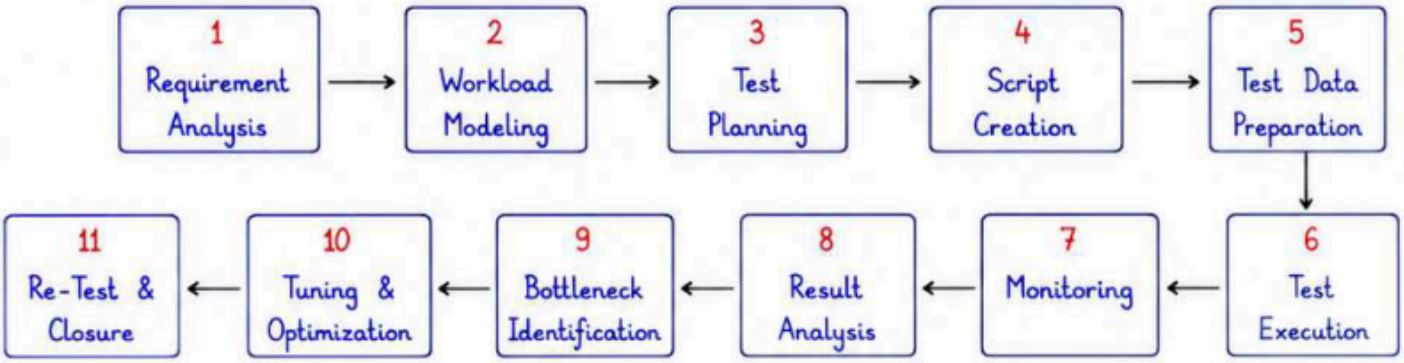
SLA : 95% of requests should have response time less than 2 seconds.
SLO : 95% of requests should be completed within 2 seconds.
SLI : P95 Response Time.

Key Metrics Relationship

Metric	What it Indicates	Used For
Response Time	How fast the system responds	User experience
Throughput (TPS)	How much work system can handle	Capacity & performance comparison
Concurrent Users (Vusers)	How many users are using at same time	Load simulation
Error Rate	How many requests are failing	System stability
Resource Utilization	How much resources are used	Bottleneck identification

5. Performance Testing Life Cycle

Performance testing is an iterative process. Each phase may lead to rework until performance goals are met.



Phase Description

1. **Requirement Analysis** : Understand business goals, SLAs, expected load, critical transactions.
2. **Workload Modeling** : Convert business scenarios into workload model and user behavior.
3. **Test Planning** : Define scope, test types, approach, tools, environment, entry & exit criteria.
4. **Script Creation** : Create scripts using LoadRunner (VuGen) to simulate user actions.
5. **Test Data Preparation** : Prepare realistic and enough data for testing.
6. **Test Execution** : Execute tests using LoadRunner Controllor with virtual users.
7. **Monitoring** : Monitor application / server / resources during test execution.
8. **Result Analysis** : Analyze results, metrics and compare with SLA.
9. **Bottleneck Identification** : Identify performance bottlenecks in application or infrastructure.
10. **Tuning & Optimization** : Fix issues, optimize code / configuration / infrastructure.
11. **Re-Test & Closure** : Re-run tests to validate fixes and close the cycle.

Key Points

- ★ Performance testing is not a one-time activity. It is iterative.
- ★ Early performance testing helps in identifying issues early.
- ★ Good monitoring + analysis = Better performance.
- ★ Performance Testing is a team effort (Dev, QA, Ops, DBAs, Network).

Entry & Exit Criteria

Entry Criteria:

- Environment ready
- Build is stable
- Test data ready
- Scripts reviewed

Exit Criteria:

- All test scenarios executed
- SLA targets achieved
- No critical issues open

LoadRunner Perspective

- → **VuGen** → Create and maintain scripts.
- → **Controller** → Schedule and execute load tests.
- → **Analysis** → Analyze results and generate reports.

6. Workload & Test Scenario Design

Good workload design = Realistic test = Meaningful results.

Key Factors in Workload Design

- Business Transactions : Identify critical user journeys.
- User Distribution : How users are distributed across transactions.
- Concurrent Users (Vusers) : Number of users performing at the same time.
- Arrival Rate / Throughput : Number of requests or transactions per second.
- Think Time : Time between two user actions.
- Pacing : Constant / Step / Ramped / Custom.
- Ramp-up Period : Time to gradually increase load.
- Steady State : Time when load remains constant.
- Ramp-down Period : Time to gradually decrease load.
- Test Duration : Total time for which test will run.
- Peak-Hour Traffic : Test should reflect peak business hours.
- Production Like Workload : Data, environment, mix of transactions should be realistic.

Users vs Concurrent Users

Total Users in a day may be 10,000 but they are not online at the same time.

We test based on Concurrent Users (Vusers).

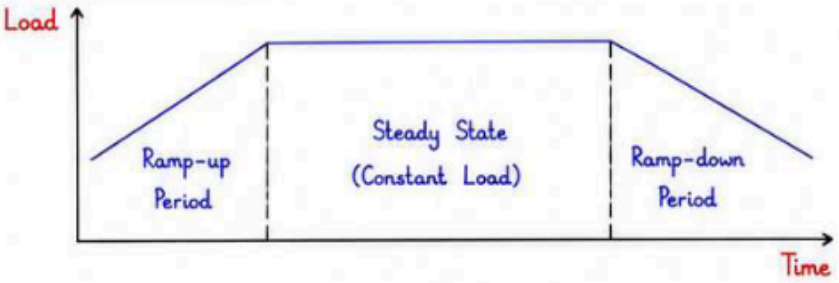
Example:

10,000 users/day

Peak time : 10% users active

→ 1,000 concurrent users (approx)

Typical Load Pattern



Pacing Types

- ★ Constant Pacing – Users start together
- ★ Step Pacing – Users increase in steps
- ★ Ramped Pacing – Users increase gradually
- ★ Custom Pacing – As per custom pattern

Example – Converting Business Traffic to Test Scenario

An e-commerce application has below stats:

- Total users per day = 10,000
- Peak hours = 2 hours (8 PM – 10 PM)
- % users active in peak time = 10%
- Avg think time = 5 seconds
- Avg requests per user = 20 requests in 2 hours

Step 1: Calculate Concurrent Users (Vusers)

Concurrent Users = Total Users × Peak %

= 10,000 × 10% = 1,000 users

Step 2: Calculate Arrival Rate (Requests per Second)

Total requests in peak time = 1,000 users × 20 = 20,000 requests

Duration = 2 hours = 7200 seconds

Arrival Rate = Total Requests / Duration

= 20,000 / 7200 = 2.78 ≈ 3 requests/sec

Test Scenario

- ★ Virtual Users (Concurrent) : 1,000
- ★ Ramp-up Period : 15 minutes
- ★ Steady State Period : 90 minutes
- ★ Ramp-down Period : 15 minutes
- ★ Total Duration : 120 minutes
- ★ Arrival Rate (Throughput) : 3 requests/sec
- ★ Think Time : 5 seconds
- ★ Pacing : Ramped
- ★ Environment : Production like
- ★ Data : Realistic test data
- ★ Transactions Mix : As per production
- ★ SLA : 95% requests < 2 sec (P95)

Note: ★ The goal is not to hit a number of users but to simulate real business behavior.

7. LoadRunner Introduction & Architecture

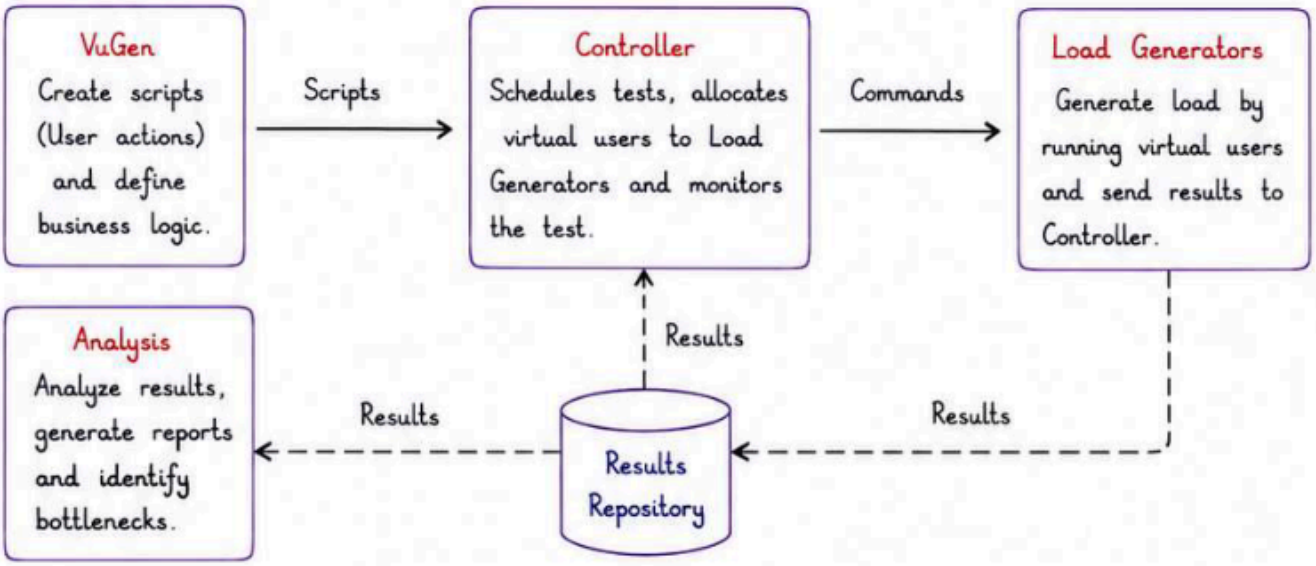
What is LoadRunner ?

- LoadRunner is a performance testing tool from OpenText.
- It helps to test the performance, reliability and scalability of applications under heavy load.
- It allows you to simulate thousands of users (Virtual Users) accessing the application at the same time.
- Supports a wide range of protocols and technologies.
- Main components: VuGen, Controller, Load Generators, Analysis.

Why LoadRunner ?

- ★ Industry standard performance testing tool.
- ★ Supports enterprise applications.
- ★ Can simulate large number of users.
- ★ Detailed analysis and reporting.
- ★ Integrates with monitoring tools.

LoadRunner Architecture



Key LoadRunner Components

Component	Purpose	Description
VuGen	Script creation tool	Used to create and maintain performance test scripts. Simulates user actions on the application.
Controller	Test management & execution	Schedules the load test, allocates virtual users to load generators and monitors the test progress.
Load Generators	Load generation machines	Machines that generate the load by running virtual users and send results to the Controller.
Analysis	Result analysis & reporting	Provides graphs, reports and dashboards to analyze performance metrics and identify issues.
Results Repository	Store test results	Central storage where all test results are collected and stored for analysis.

8. VuGen & Creating a Performance Script

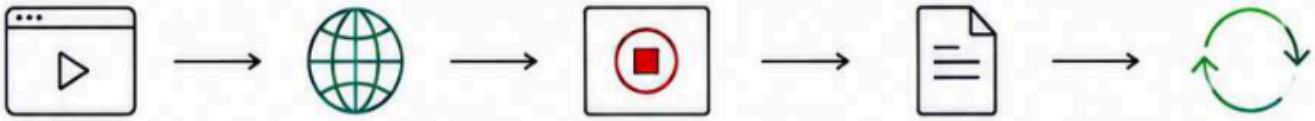
What is VuGen ?

- VuGen (Virtual User Generator) is used to create and maintain performance test scripts.
- It simulates user actions (Vusers) on the application.
- Scripts are written in C language.
- Supported protocols: Web, HTTP/HTTPS, Web Services, SAP, Java, .NET, Citrix, etc.

Steps to Create Script in VuGen

1. Start VuGen → New Script
2. Select the protocol (e.g., Web - HTTP/HTML)
3. Configure Recording Options
4. Start Recording
5. Perform user actions on the application
6. Stop Recording
7. Save the Script
8. Replay the Script (Validate)

VuGen - Script Creation Flow



1. Start Recording
   Start recording user actions.
2. Perform Actions
   Perform actions on the application.
3. Stop Recording
   Stop recording.
4. Save Script
   Save the recorded script.
5. Replay Script
   Replay and validate the script.

Recording Modes in VuGen

- Analog Recording : Records all the details (default).
- Broadband Recording : Records only necessary details, reduces script size.
- Extended Recording : Records more details, useful for troubleshooting.

Important Options Before Recording

- Select protocol and technology.
- Choose recording mode.
- Set options: Think Time, Timeout, Transactions.
- Choose how to handle pop-ups, warnings, and errors.

Think Time

- Time between two user actions.
- Makes the script realistic.
- Can be set during recording or added later.

Types of Scripts

Script Type	Description	Use
Action	Single user script.	Functional + Performance testing
Business Process	Combination of multiple actions.	End-to-end business scenario
Web Services	Web service based applications.	SOAP / REST services testing
Unit / C Unit	Protocol level scripts.	Component testing

9. Load Generators – Generating the Load

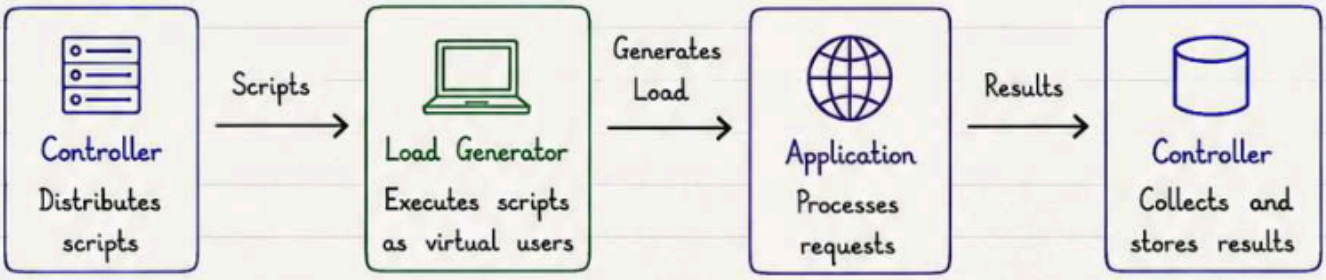
What is Load Generator ?

- Load Generator is a component that simulates real users (virtual users).
- It executes the scripts assigned by the Controller.
- It generates load on the application and sends results back to Controller.
- Multiple Load Generators can run in parallel to create heavy load.

Key Responsibilities

- ★ Receive scripts from Controller.
- ★ Execute the scripts as virtual users.
- ★ Generate load on the application.
- ★ Collect metrics and results.
- ★ Send results back to Controller.

How Load Generator Works ?



Load Generator Settings

- Number of Users : Number of virtual users to simulate.
- Ramp-Up Time : Time taken to reach the target users.
- Loop Count : Number of iterations for the script.
- Think Time : Delay between user actions.
- Pacing : Constant / Unlimited / Once / Per Iteration.
- Monitoring : Enable detailed performance metrics.
- Log Settings : Log level and log file location.

Note

- Think time makes the test realistic.
- Ramp-up helps to avoid sudden load on the system.
- More Load Generators = More load.

Types of Load Generators

Type	Description	When to Use
Local Load Generator	Runs on the same machine where VuGen is installed.	For small load tests or learning.
Remote Load Generator	Runs on a different machine.	For medium to large load tests.
Cloud / Grid Load Generator	Runs on cloud or distributed grid.	For very high load and enterprise tests.

Best Practices

- ★ Distribute load across multiple Load Generators.
- ★ Ensure proper network bandwidth and low latency.
- ★ Monitor CPU, Memory, and Network of Load Generators.
- ★ Keep Load Generators time synchronized with Controller.
- ★ Use dedicated machines for Load Generators.

10. Scenarios in LoadRunner

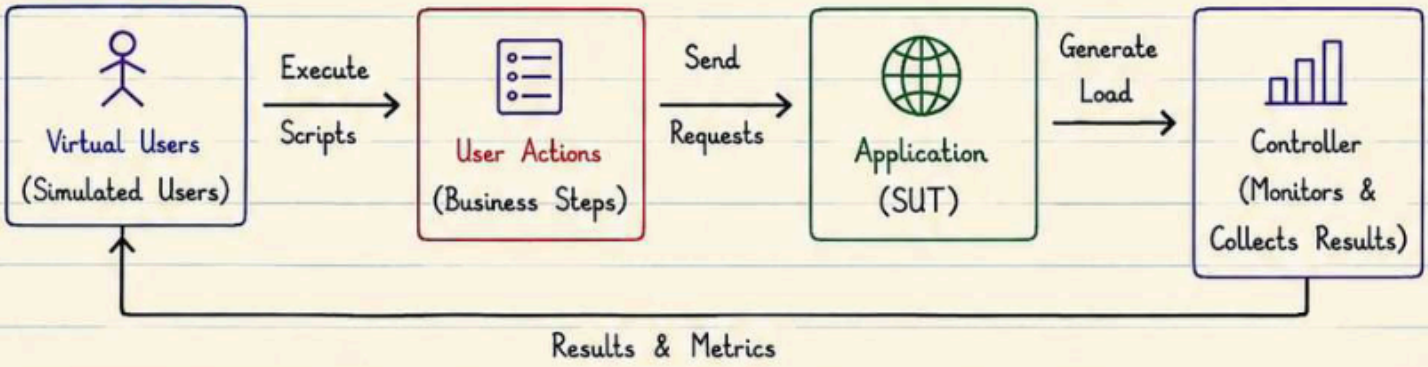
What is a Scenario ?

- Scenario defines user behavior during the test.
- It includes what actions users will perform and how many users will perform them.
- Scenarios are created using Controllers.
- Good scenarios = Realistic test.

Common Scenario Types

- ★ Basic Scenario
- ★ Goal-Oriented Scenario
- ★ Multi-Step Scenario
- ★ Business Process Scenario
- ★ Stress Scenario
- ★ Soak / Endurance Scenario

Scenario Flow



Example Scenario

Step	Action	Virtual Users	Ramp-Up	Duration	Think Time
1	Login to Application	100	2 min	5 min	3 sec
2	Search Product	100	–	5 min	2 sec
3	Add to Cart	100	–	3 min	2 sec
4	Checkout	100	–	5 min	3 sec
Total Users		100	–	15 min	–

Tips for Good Scenarios

- Understand business process.
- Include all important user actions.
- Set realistic think time.
- Define proper ramp-up.
- Use parameterization for dynamic data.
- Start small, then scale.
- Validate scripts before running scenarios.

Notes

- Scenario should mimic real user behavior.
- Avoid constant load without think time.
- More realistic scenarios = Better results.
- Always run baseline scenario first.

11. Analyzing Results in LoadRunner

Where to Analyze ?

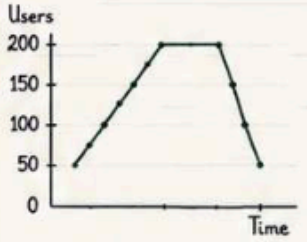
- After the test run, results are stored in the Results Directory.
- Open Analysis in LoadRunner Controller to view graphs, tables and reports.
- Helps us to identify performance bottlenecks and system behavior.

Key Metrics

- ★ Vuser Hits : Total number of requests.
- ★ Transactions : Successful business transactions.
- ★ Average Response Time : Avg. time for a transaction.
- ★ Throughput : Number of transactions per second.
- ★ Errors : Number of failed transactions.
- ★ Bandwidth : Network usage during the test.

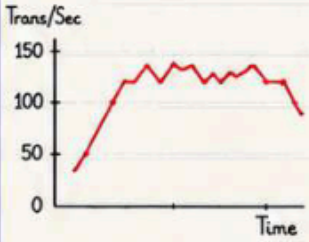
Important Graphs

1. Running Vusers



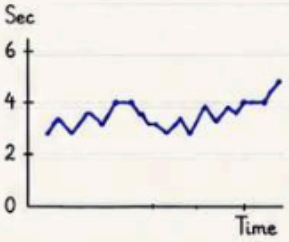
Shows how many Vusers were active during the test.

2. Throughput



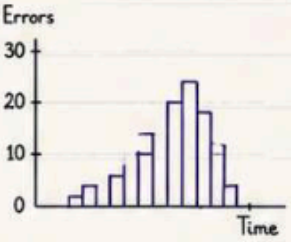
Shows how many transactions are processed per second.

3. Response Time



Shows average response time of transactions.

4. Errors



Shows number of errors occurred during the test.

Steps to Analyze Results

1. **Open Analysis** — Open the results in Analysis tool.
2. **Select Graphs** — Choose graphs and metrics to view.
3. **Identify Issues** — Look for high response time, errors, low throughput.
4. **Drill Down** — Drill down to transaction or request level.
5. **Generate Report** — Create and share custom reports.

Correlation in LoadRunner

- Correlation is used to extract dynamic data from server response and use it in next requests.
- Example: Session ID, Token, Order ID, etc.
- Steps:
  1. Identify dynamic data in response.
  2. Create a correlation rule.
  3. Parameterize the value.
  4. Use the parameter in next request.

Example

Response: "sessionId": "abc123xyz"

↓

Create parameter: {sessionId}

↓

Use in next request body or header.

Quick Tips

- ★ Keep think time realistic.
- ★ Use realistic data and scenarios.
- ★ Monitor server resources (CPU, Memory, Network).
- ★ Start with small load and gradually increase.
- ★ Always run baseline test first.
- ★ Analyze, identify bottlenecks and optimize.
- ★ Re-run test after fixes and compare results.

12. Parameterization in LoadRunner

What is Parameterization ?

- Parameterization allows you to replace static data with dynamic values.
- It simulates real user behavior with different data.
- Helps in avoiding duplicate records and test data conflicts.

Benefits

- ★ Simulates multiple users with unique data.
- ★ Prevents data overlap and duplication.
- ★ Increases test coverage.
- ★ Improves test accuracy and reliability.
- ★ Supports dynamic and runtime data.

Parameterization Methods in LoadRunner

Method	Description	When to Use	Example
1. Correlation	Extracts dynamic values from server response and uses in next requests.	When server generates values (Session ID, Token, Order ID).	`web_reg_save_param("sessionId", "LB=user_session=", "RB=;\'");`
2. Data Driven	Uses data from external files (Excel, CSV) during runtime.	For multiple sets of input data (login, search, checkout).	`web_set_sockets_option("DDE", ...);` `lr_paramarr_random("user",1);`
3. Parameter List	Uses parameter list created in LoadRunner.	For simple sets of data managed in VuGen.	`web_reg_find("Text=","SaveCount=cnt");` `lr_paramarr_idx("param", cnt);`
4. Runtime Settings	Passes parameter values from command line at runtime.	When values change per run (Environment, User, etc.).	`lr_get_param("Env", "Prod");`
5. Functions	Uses built-in functions to generate dynamic data.	For dates, random numbers, strings, etc.	`lr_eval_string("{=%random%}");` `lr_eval_string("{=StringLen}");`

Correlation Example

Scenario:
Extract sessionId from login response and use it in next request.

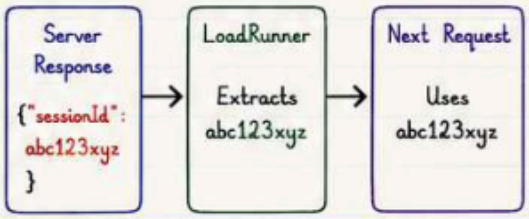
Step 1: Save Parameter

```
web_reg_save_param("sessionId",
    "LB="sessionId":\"","RB=\",
    SEARCH_FILTERS,
    "Scope=Body",
    LAST);
```

Step 2: Use Parameter

```
web_custom_request("Dashboard",
    "URL=/dashboard",
    "Method=GET",
    "Resource=0",
    "RecContentType=text/html",
    "Snapshot=t1.inf",
    "Mode=HTML",
    LAST);
web_add_auto_header("sessionId","{sessionId}");
```

How Correlation Works ?



Data Driven Example

username	password
user1	pass@123
user2	pass@123
user3	pass@123
...	...

Steps:

① Create CSV file with test data.
② Configure Data Source in Runtime Settings.
③ Use lr_paramarr_random() or lr_paramarr_idx().
④ Run test to iterate through data.

Sample CSV File (users.csv)

```
username,password
user1,pass@123
user2,pass@123
user3,pass@123
...
```

Best Practices

- ★ Always correlate dynamic values.
- ★ Use parameterization for all input fields.
- ★ Avoid hardcoding data in scripts.
- ★ Validate parameter values before using.
- ★ Use realistic and unique data.

Notes

- Start with correlation, then parameterization.
- Always run baseline test before load test.
- Keep parameter files up to date.
- Review server responses for correct correlation.

13. Assertions in LoadRunner

@ Aman_Views

What is Assertion ?

- Assertion is used to validate that the application is behaving as expected.
- It compares actual response with expected values or conditions.
- Helps in identifying functional issues during the test.
- Failed assertions are marked as errors in results.

Why Use Assertions ?

★ Validates business logic.

★ Ensures data integrity.

★ Detects application errors early.

★ Increases test reliability.

★ Helps in better reporting.

Types of Assertions

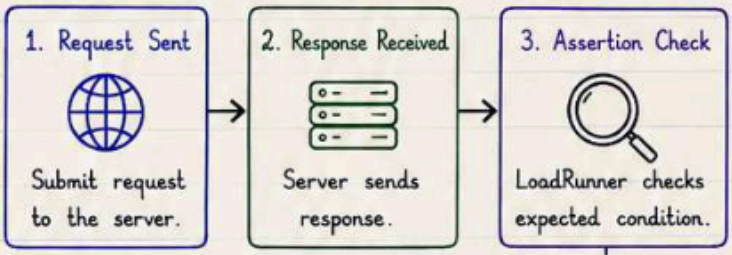
Type	Description	Example
1. String Assertion	Validates that a specific string exists in the response.	`web_reg_find("Text=Welcome");`
2. Text/IC Check	Validates text or values at a specific location.	`web_reg_save_param_ex("ParamName=Title", "LB=", "RB=</title>");`
3. Boundary Assertion	Validates data between left and right boundaries.	`web_reg_save_param("LB=<span id='price'>", "RB=</span>", "ParamName=Price");`
4. Content Check	Checks content type, status code, or other headers.	`web_reg_add_header("Content-Type", "application/json");`

Assertion Example (String Assertion)

```
web_reg_find("Text=Thank You",
             LAST);

web_submit_data("SubmitOrder",
  "Action=http://testapp.com/order",
  "Method=POST",
  ITEMDATA,
  "Name=product", "Value=Book", ENDITEM,
  "Name=qty", "Value=1", ENDITEM,
  LAST);
```

How Assertion Works in Script



Result

- If assertion is TRUE → Pass
- If assertion is FALSE → Fail (Error)

Common Assertion Use Cases

Use Case	Example
Login Validation	Verify welcome message after successful login.
Error Message Validation	Check for specific error message on invalid input.
Total Count Validation	Validate total number of records returned.
Data Integrity	Ensure important data is present and correct.
Page Title Validation	Verify page title after navigation.

Tips

★ Use meaningful assertion messages.

★ Don't overuse assertions.

★ Keep assertions simple and specific.

★ Review failed assertions in analysis.